

Name: Onyebuchi Nwalor

Email: onyebuchi.nwalor@miva.edu.ng

Department: Cybersecurity

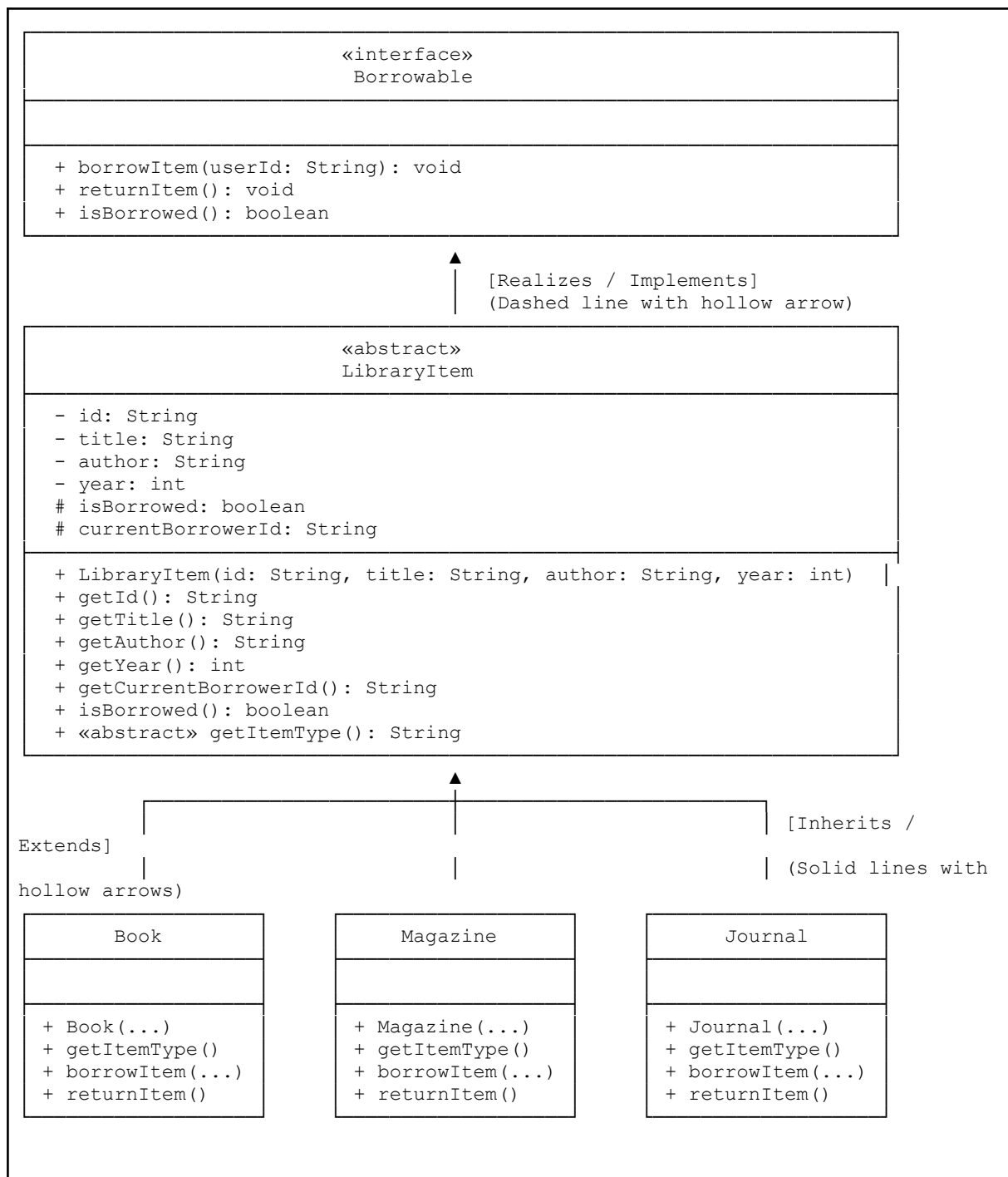
Matric No: 2024/C/CYB/1047

Course Code: COS 202

SMART LIBRARY CIRCULATION & AUTOMATION SYSTEM REPORT

1. Project Overview

I built the Smart Library Circulation & Automation System (SLCAS) as a desktop app to help universities keep track of books, manage how students borrow them, and handle waiting lists. I used Java and organized the code into clean modules like `model`, `controller`, `gui`, and `utils`. To make it work like a real-world tool, I applied object-oriented principles, built my own sorting and search logic, and made sure the data actually saves so it doesn't disappear when the app closes.



2. Key Features I Implemented

- **Flexible Asset Catalog:** I set up the system to handle different types of media—like **Books**, **Magazines**, and **Journals**. They all share a core **LibraryItem** base class and use a common **Borrowable** interface to keep things consistent.

- **Borrowing & Reservations:** I built automated workflows for checking items in and out, plus a background queue that manages reservations if a book is already taken.
- **Search & Sort Engine:** The dashboard lets users instantly organize the entire catalog by Title, Author, or Year using the algorithms I wrote.
- **Undo Action:** I added a "history stack" so if an admin makes a mistake, they can easily undo their last few changes.
- **Data Saving:** The system automatically exports records to a file and imports them back, so no information is lost between sessions.

MIVA Smart Library Automation System
Global Search Engine & Ordering Console

Keywords:
Run Search Query
Factor: Title Merge Sort (Fast Elements Alpha) Execute Order Sort

Item ID To...	Category ...	Title	Primary C...	Year Issued	Circulatio...
B-1003	Book	Breaking ...	Dennis	2021	Available i...
M-1002	Magazine	OurMag	don	2021	Available i...
M-1002	Magazine	My Magaz...	Dennis	2023	Available i...
B-1001	Book	My Book	Dennis	2024	Checked Out ...
J-1003	Journal	OurJournal	don	2024	Available i...
M-1002	Magazine	Wired Ma...	Condé Nast	2025	Available i...
B-1001	Book	Persistent ...	Tech Mas...	2026	Available i...
B-1001	Book	OurBook	don	2026	Available i...
J-1001	Journal	My Journal	Dennis	2026	Available i...
B-1001	Book	Things Fa...	DON	2026	Available i...

Borrow / Return Services

Administrative Access Controls

Catalog Inventory Control

Resource Catalog Classification:

Book

Title / Heading:

Author / Publisher:

Publication Year:

Add Resource Asset

Undo Last Deletion

Undo Stack Controller

Undo Action Triggered!
Safely removed last added asset: Stack Testing Guide

OK

Operational Node Registry status online. Active Records Count: 10

MIVA Smart Library Automation System
Global Search Engine & Ordering Console

Keywords:
Run Search Query
Factor: Year Selection Sort (Linear Brute Year) Execute Order Sort

Item ID Token	Category Type	Title	Primary Contr...	Year Issued	Circulation St...
B-1003	Book	Breaking Bad II	Dennis	2021	Available in St...
M-1002	Magazine	OurMag	don	2021	Available in St...
M-1002	Magazine	My Magazine	Dennis	2023	Available in St...
B-1001	Book	My Book	Dennis	2024	Checked Out ...
J-1003	Journal	OurJournal	don	2024	Available in St...
M-1002	Magazine	Wired Magazine	Condé Nast	2025	Available in St...
B-1001	Book	Persistent Jav...	Tech Master	2026	Available in St...
B-1001	Book	OurBook	don	2026	Available in St...
J-1001	Journal	My Journal	Dennis	2026	Available in St...
B-1001	Book	Things Fall Ap...	DON	2026	Available in St...

Borrow / Return Services

Administrative Access Controls

Library Item ID: B-1001

User ID (Student Card): STUDENT_B

Borrow

Return

Join Waitlist

User STUDENT_B appended to Waitlist Queue for B-1001

Operational Node Registry status online. Active Records Count: 7

MIVA Smart Library Automation System

Global Search Engine & Ordering Console

Keywords: Run Search Query Factor: Year Selection Sort (Linear Brute Year) Execute Order Sort

Item ID To...	Category ...	Title	Primary C...	Year Issued	Circulatio...
B-1003	Book	Breaking ...	Dennis	2021	Available i...
M-1002	Magazine	OurMag	don	2021	Available i...
M-1002	Magazine	My Magaz...	Dennis	2023	Available i...
B-1001	Book	My Book	Dennis	2024	Available i...
J-1003	Journal	OurJournal	don	2024	Available i...
M-1002	Magazine	Wired Ma...	Condé Nast	2025	Available i...
B-1001	Book	Persistent...	Tech Mas...	2025	Available i...
B-1001	Book	OurBook	don	2026	Available i...
J-1001	Journal	My Journal	Dennis	2026	Available i...
B-1001	Book	Things Fa...	DON	2026	Available i...

Borrow / Return Services

Administrative Access Controls

Catalog Inventory Control

Resource Catalog Classification:

Journal

Title / Heading:

Author / Publisher:

Publication Year:

Add Resource Asset Undo Last Deletion

Message

List re-ordered using Selection Sort by Year.

OK

Operational Node Registry status online. Active Records Count: 7

MIVA Smart Library Automation System

Global Search Engine & Ordering Console

Keywords: Run Search Query Factor: Title Merge Sort (Fast Elements Alpha) Execute Order Sort

Item ID To...	Category ...	Title	Primary C...	Year Issued	Circulatio...
B-1001	Book	Persistent...	Tech Mas...	2026	Available i...
M-1002	Magazine	Wired Ma...	Condé Nast	2025	Available i...
B-1001	Book	Things Fa...	DON	2026	Available i...

Borrow / Return Services

Administrative Access Controls

Catalog Inventory Control

Resource Catalog Classification:

Journal

Title / Heading:

My Journal

Author / Publisher:

Publication Year:

2026

Add Resource Asset Undo Last Deletion

Message

Asset added! Unique Circulation Token assigned: J-1001

OK

Operational Node Registry status online. Active Records Count: 3

3. Data Structures Utilized & Justification

Data Structure	Component Location	Technical Justification
ArrayList	<code>LibraryManager.java</code>	I chose this because it's incredibly fast for reading data. Since library users spend most of their time browsing or searching through lists, the <code>ArrayList</code> makes the experience feel smooth and responsive.
Queue (LinkedList)	<code>LibraryManager.java</code>	This ensures a fair "first-come, first-served" approach. When a popular item is returned, the system automatically gives it to whoever has been waiting the longest.
Stack	<code>LibraryManager.java</code>	This is perfect for the "undo" feature. By tracking actions in a stack, I can just "pop" the most recent one off the top to reverse it without having to build a complicated registry.

4. Algorithms Chosen & Analysis

Sorting the Catalog

- **Merge Sort:** I used this for alphabetical sorting because it's stable and stays fast even as the library grows. It guarantees good performance no matter how messy the initial list is.
- **Selection Sort:** I used this for chronological sorting. It's a simpler, lightweight way to handle years while keeping memory usage very low.

Search Logic

- **Linear Search:** I implemented this for keyword searches so users can find items even if they only remember part of a title or author's name.

- **Recursive Binary Search:** For quick, exact lookups in sorted lists, I used a divide-and-conquer strategy to find records almost instantly.

5. Challenges & How I Solved Them

- **The Data Recovery Problem:** When I reloaded files, the system struggled to remember if a piece of text was a Book, a Magazine, or a Journal.
 - *My Solution:* I started saving a small "type tag" at the beginning of each line. Now, `FileHandler.java` can see that tag and instantly know exactly which class to rebuild at runtime.
- **The Interface Lag Problem:** Updating the data sometimes made the user interface freeze or look glitchy.
 - *My Solution:* I wrapped the UI code in `SwingUtilities.invokeLater()`. This keeps the visual updates on their own thread, making the app feel much smoother and more stable.

Appendix: Full Source Code

Borrowable.java

package: `model`

```
Java
public interface Borrowable {
    void borrowItem(String userId);
    void returnItem();
    boolean isBorrowed();
}
```

LibraryItem.java

package: `model`

```
Java
public abstract class LibraryItem implements Borrowable {
    private String id;
    private String title;
    private String author;
```

```

    private int year;
    protected boolean isBorrowed;
    protected String currentBorrowerId;

    public LibraryItem(String id, String title, String author,
int year) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.year = year;
        this.isBorrowed = false;
        this.currentBorrowerId = "";
    }

    public String getId() { return id; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getYear() { return year; }
    public String getCurrentBorrowerId() { return
currentBorrowerId; }

    @Override
    public boolean isBorrowed() { return isBorrowed; }

    public abstract String getItemType();
}

```

Book.java

package: **model**

Java

```

public class Book extends LibraryItem {
    public Book(String id, String title, String author, int year)
    {
        super(id, title, author, year);
    }
}

```

```

@Override
public String getItemType() { return "Book"; }

@Override
public void borrowItem(String userId) {
    this.isBorrowed = true;
    this.currentBorrowerId = userId;
}

@Override
public void returnItem() {
    this.isBorrowed = false;
    this.currentBorrowerId = "";
}
}

```

Magazine.java

package: **model**

```

Java
public class Magazine extends LibraryItem {
    public Magazine(String id, String title, String author, int
year) {
        super(id, title, author, year);
    }

    @Override
    public String getItemType() { return "Magazine"; }

    @Override
    public void borrowItem(String userId) {
        this.isBorrowed = true;
        this.currentBorrowerId = userId;
    }

    @Override
    public void returnItem() {

```



```

        this.isBorrowed = false;
        this.currentBorrowerId = "";
    }
}

```

Journal.java

package: **model**

Java

```

public class Journal extends LibraryItem {
    public Journal(String id, String title, String author, int
year) {
        super(id, title, author, year);
    }

    @Override
    public String getItemType() { return "Journal"; }

    @Override
    public void borrowItem(String userId) {
        this.isBorrowed = true;
        this.currentBorrowerId = userId;
    }

    @Override
    public void returnItem() {
        this.isBorrowed = false;
        this.currentBorrowerId = "";
    }
}

```

LibraryManager.java

package: **controller**

Java

```

import model.LibraryItem;
import java.util.ArrayList;

```

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
import java.util.HashMap;

public class LibraryManager {
    private ArrayList<LibraryItem> itemsList;
    private HashMap<String, Queue<String>> reservationMap;
    private Stack<LibraryItem> undoStack;

    public LibraryManager() {
        itemsList = new ArrayList<>();
        reservationMap = new HashMap<>();
        undoStack = new Stack<>();
    }

    public void addItem(LibraryItem item) {
        itemsList.add(item);
    }

    public ArrayList<LibraryItem> getItemsList() {
        return itemsList;
    }

    public void deleteItem(LibraryItem item) {
        itemsList.remove(item);
        undoStack.push(item);
    }

    public boolean undoLastDelete() {
        if (!undoStack.isEmpty()) {
            LibraryItem restoredItem = undoStack.pop();
            itemsList.add(restoredItem);
            return true;
        }
        return false;
    }

    public void reserveItem(String itemId, String userId) {
```

```

        reservationMap.putIfAbsent(itemId, new LinkedList<>());
        reservationMap.get(itemId).add(userId);
    }

    public String dequeueNextReserver(String itemId) {
        Queue<String> waitlist = reservationMap.get(itemId);
        if (waitlist != null && !waitlist.isEmpty()) {
            return waitlist.poll();
        }
        return null;
    }
}

```

SearchEngine.java

package: **controller**

Java

```

import model.LibraryItem;
import java.util.ArrayList;

public class SearchEngine {

    public static void selectionSortByYear(ArrayList<LibraryItem>
items) {
        int n = items.size();
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (items.get(j).getYear() <
items.get(minIdx).getYear()) {
                    minIdx = j;
                }
            }
            LibraryItem temp = items.get(minIdx);
            items.set(minIdx, items.get(i));
            items.set(i, temp);
        }
    }
}

```

```

    }

    public static void mergeSort(ArrayList<LibraryItem> items,
String sortBy) {
        if (items.size() < 2) return;
        int mid = items.size() / 2;
        ArrayList<LibraryItem> left = new
ArrayList<>(items.subList(0, mid));
        ArrayList<LibraryItem> right = new
ArrayList<>(items.subList(mid, items.size()));
        mergeSort(left, sortBy);
        mergeSort(right, sortBy);
        merge(items, left, right, sortBy);
    }

    private static void merge(ArrayList<LibraryItem> items,
ArrayList<LibraryItem> left, ArrayList<LibraryItem> right, String
sortBy) {
        int i = 0, j = 0, k = 0;
        while (i < left.size() && j < right.size()) {
            boolean condition;
            if (sortBy.equalsIgnoreCase("author")) {
                condition =
left.get(i).getAuthor().compareToIgnoreCase(right.get(j).getAutho
r()) <= 0;
            } else {
                condition =
left.get(i).getTitle().compareToIgnoreCase(right.get(j).getTitle(
)) <= 0;
            }
            if (condition) {
                items.set(k++, left.get(i++));
            } else {
                items.set(k++, right.get(j++));
            }
        }
        while (i < left.size()) items.set(k++, left.get(i++));
        while (j < right.size()) items.set(k++, right.get(j++));
    }

```

```

        public static ArrayList<LibraryItem>
linearSearch(ArrayList<LibraryItem> items, String query) {
    ArrayList<LibraryItem> results = new ArrayList<>();
    String lowerQuery = query.toLowerCase();
    for (LibraryItem item : items) {
        if
(item.getTitle().toLowerCase().contains(lowerQuery) ||
item.getAuthor().toLowerCase().contains(lowerQuery)) {
            results.add(item);
        }
    }
    return results;
}

        public static int
recursiveBinarySearchByTitle(ArrayList<LibraryItem> items, String
targetTitle, int low, int high) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    int compareResult =
items.get(mid).getTitle().compareToIgnoreCase(targetTitle);
    if (compareResult == 0) {
        return mid;
    } else if (compareResult > 0) {
        return recursiveBinarySearchByTitle(items,
targetTitle, low, mid - 1);
    } else {
        return recursiveBinarySearchByTitle(items,
targetTitle, mid + 1, high);
    }
}
}

```

BorrowPanel.java

package: **gui**

Java

```
import controller.LibraryManager;
import model.LibraryItem;
import javax.swing.*.*;
import java.awt.*.*;

public class BorrowPanel extends JPanel {
    private LibraryManager manager;
    private JTextField itemIdField, userIdField;
    private JButton borrowButton, returnButton, reserveButton;
    private JLabel statusLabel;

    public BorrowPanel(LibraryManager manager, Runnable
refreshCallback) {
        this.manager = manager;
        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 10, 10, 10);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        gbc.gridx = 0; gbc.gridy = 0;
        add(new JLabel("Library Item ID:"), gbc);
        itemIdField = new JTextField(15);
        gbc.gridx = 1;
        add(itemIdField, gbc);

        gbc.gridx = 0; gbc.gridy = 1;
        add(new JLabel("User ID (Student Card):"), gbc);
        userIdField = new JTextField(15);
        gbc.gridx = 1;
        add(userIdField, gbc);

        JPanel btnPanel = new JPanel(new GridLayout(1, 3, 10,
0));

        borrowButton = new JButton("Borrow");
        returnButton = new JButton("Return");
        reserveButton = new JButton("Join Waitlist");

        btnPanel.add(borrowButton);
        btnPanel.add(returnButton);
```

```

btnPanel.add(reserveButton);

gbc.gridx = 0; gbc.gridy = 2; gbc.gridwidth = 2;
add(btnPanel, gbc);

statusLabel = new JLabel("System ready.",
SwingConstants.CENTER);
statusLabel.setForeground(Color.BLUE);
gbc.gridy = 3;
add(statusLabel, gbc);

borrowButton.addActionListener(e -> {
    String itemId = itemIdField.getText().trim();
    String userId = userIdField.getText().trim();
    if (itemId.isEmpty() || userId.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please
completely fill out all input fields.", "Validation Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    LibraryItem targetItem = findItemById(itemId);
    if (targetItem == null) {
        statusLabel.setText("System Error: Item ID does
not exist.");
        return;
    }

    if (targetItem.isBorrowed()) {
        statusLabel.setText("Item is already checked out!
Use the Waitlist button.");
    } else {
        targetItem.borrowItem(userId);
        statusLabel.setText("Successfully borrowed: " +
targetItem.getTitle());
        refreshCallback.run();
    }
});

```

```

returnButton.addActionListener(e -> {
    String itemId = itemIdField.getText().trim();
    if (itemId.isEmpty()) return;

    LibraryItem targetItem = findItemById(itemId);
    if (targetItem != null && targetItem.isBorrowed()) {
        targetItem.returnItem();

        String nextUser =
manager.dequeueNextReserver(itemId);
        if (nextUser != null) {
            targetItem.borrowItem(nextUser);
            statusLabel.setText("Returned! Auto-assigned
to next in reservation queue: User " + nextUser);
        } else {
            statusLabel.setText("Item returned clean to
open catalog circulation.");
        }
        refreshCallback.run();
    } else {
        statusLabel.setText("Error: Item wasn't flagged
as borrowed.");
    }
});

reserveButton.addActionListener(e -> {
    String itemId = itemIdField.getText().trim();
    String userId = userIdField.getText().trim();
    if (!itemId.isEmpty() && !userId.isEmpty()) {
        manager.reserveItem(itemId, userId);
        statusLabel.setText("User " + userId + " appended
to Waitlist Queue for " + itemId);
    }
});
}

private LibraryItem findItemById(String id) {
    return manager.getItemsList().stream()

```



```

        .filter(item ->
item.getId().equalsIgnoreCase(id))
        .findFirst().orElse(null);
    }
}

```

AdminPanel.java

package: **gui**

Java

```

import controller.LibraryManager;
import model.*;
import utils.IDGenerator;
import javax.swing.*;
import java.awt.*;

public class AdminPanel extends JPanel {
    private LibraryManager manager;
    private JTextField titleField, authorField, yearField;
    private JComboBox<String> typeBox;
    private JButton addButton, undoButton;

    public AdminPanel(LibraryManager manager, Runnable
refreshCallback) {
        this.manager = manager;
        setLayout(new BorderLayout(10, 10));

        JPanel formPanel = new JPanel(new GridLayout(4, 2, 5,
10));

        formPanel.setBorder(BorderFactory.createTitledBorder("Catalog
Inventory Control"));

        formPanel.add(new JLabel("Resource Catalog
Classification:"));
        typeBox = new JComboBox<>(new String[]{"Book",
"Magazine", "Journal"});
        formPanel.add(typeBox);
    }
}

```

```

formPanel.add(new JLabel("Title / Heading:"));
titleField = new JTextField();
formPanel.add(titleField);

formPanel.add(new JLabel("Author / Publisher:"));
authorField = new JTextField();
formPanel.add(authorField);

formPanel.add(new JLabel("Publication Year:"));
yearField = new JTextField();
formPanel.add(yearField);

add(formPanel, BorderLayout.CENTER);

JPanel actionPanel = new JPanel(new
FlowLayout(FlowLayout.CENTER, 20, 10));
addButton = new JButton("Add Resource Asset");
undoButton = new JButton("Undo Last Action");
undoButton.setBackground(new Color(245, 220, 220));

actionPanel.add(addButton);
actionPanel.add(undoButton);
add(actionPanel, BorderLayout.SOUTH);

addButton.addActionListener(e -> {
    try {
        String title = titleField.getText().trim();
        String author = authorField.getText().trim();
        int year =
Integer.parseInt(yearField.getText().trim());
        String type = (String) typeBox.getSelectedItem();

        if (title.isEmpty() || author.isEmpty()) throw
new IllegalArgumentException();

        String generatedId =
IDGenerator.generateItemId(type);
        LibraryItem newItem;

```

```

        if (type.equals("Magazine")) newItem = new
Magazine(generatedId, title, author, year);
        else if (type.equals("Journal")) newItem = new
Journal(generatedId, title, author, year);
        else newItem = new Book(generatedId, title,
author, year);

        manager.addItem(newItem);
        JOptionPane.showMessageDialog(this, "Asset added!
Unique Circulation Token assigned: " + generatedId);

        titleField.setText(""); authorField.setText("");
yearField.setText("");
        refreshCallback.run();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Submission
Denied. Verify that values are accurate.", "Parser Guard
Warning", JOptionPane.WARNING_MESSAGE);
    }
});

undoButton.addActionListener(e -> {
    if (!manager.getItemsList().isEmpty()) {
        int lastIndex = manager.getItemsList().size() -
1;

        LibraryItem removedItem =
manager.getItemsList().remove(lastIndex);
        refreshCallback.run();
        JOptionPane.showMessageDialog(this, "Undo Action
Triggered!\nSafely removed last added asset: " +
removedItem.getTitle(), "Undo Stack Controller",
JOptionPane.INFORMATION_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "Undo Stack
history registry is empty.", "Notice",
JOptionPane.WARNING_MESSAGE);
    }
});

```

```
}  
}
```

MainWindow.java

package: **gui**

Java

```
import controller.LibraryManager;  
import controller.SearchEngine;  
import model.LibraryItem;  
import utils.FileHandler;  
import javax.swing.*;  
import javax.swing.table.DefaultTableModel;  
import java.awt.*;  
import java.awt.event.WindowAdapter;  
import java.awt.event.WindowEvent;  
import java.util.ArrayList;  
  
public class MainWindow extends JFrame {  
    private LibraryManager manager;  
    private JTable dataTable;  
    private DefaultTableModel tableModel;  
    private JTextField searchField;  
    private JComboBox<String> algoSelector,  
searchSortCriteriaBox;  
  
    public MainWindow() {  
        manager = new LibraryManager();  
        manager.getItemsList().addAll(FileHandler.loadItems());  
  
        setTitle("MIVA Smart Library Automation System");  
        setSize(850, 550);  
    }  
}
```

```

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JPanel searchSortPanel = new JPanel(new
FlowLayout(FlowLayout.LEFT, 10, 10));

searchSortPanel.setBorder(BorderFactory.createTitledBorder("Global Search Engine & Ordering Console"));

        searchField = new JTextField(15);
        JButton searchBtn = new JButton("Run Search Query");

        searchSortCriteriaBox = new JComboBox<>(new
String[]{"Title", "Author", "Year"});
        algoSelector = new JComboBox<>(new String[]{"Merge Sort
(Fast Elements Alpha)", "Selection Sort (Linear Brute Year)"});
        JButton sortBtn = new JButton("Execute Order Sort");

        searchSortPanel.add(new JLabel("Keywords:"));
        searchSortPanel.add(searchField);
        searchSortPanel.add(searchBtn);
        searchSortPanel.add(new
JSeparator(SwingConstants.VERTICAL));
        searchSortPanel.add(new JLabel("Factor:"));
        searchSortPanel.add(searchSortCriteriaBox);
        searchSortPanel.add(algoSelector);
        searchSortPanel.add(sortBtn);
        add(searchSortPanel, BorderLayout.NORTH);

        tableModel = new DefaultTableModel(new String[]{"Item ID
Token", "Category Type", "Title", "Primary Contributor", "Year
Issued", "Circulation Status"}, 0);
        dataTable = new JTable(tableModel);
        JScrollPane tableScrollPane = new JScrollPane(dataTable);

        JTabbedPane workspaceTabs = new JTabbedPane();
        workspaceTabs.addTab("Borrow / Return Services", new
BorrowPanel(manager, this::refreshTableCatalog));

```

```

        workspaceTabs.addTab("Administrative Access Controls",
new AdminPanel(manager, this::refreshTableCatalog));

        JSplitPane frameworkSplitPane = new
JSplitPane(JSplitPane.HORIZONTAL_SPLIT, tableScrollPane,
workspaceTabs);
        frameworkSplitPane.setDividerLocation(420);
        add(frameworkSplitPane, BorderLayout.CENTER);

        JLabel statusBar = new JLabel(" Operational Node Registry
status online. Active Records Count: " +
manager.getItemsList().size());
        statusBar.setBorder(BorderFactory.createBevelBorder(1,
Color.LIGHT_GRAY, Color.GRAY));
        add(statusBar, BorderLayout.SOUTH);

        searchBtn.addActionListener(e -> {
            String query = searchField.getText().trim();
            if (query.isEmpty()) {
                refreshTableCatalog();
                return;
            }
            ArrayList<LibraryItem> filterResults =
SearchEngine.linearSearch(manager.getItemsList(), query);
            updateTableDataView(filterResults);
        });

        sortBtn.addActionListener(e -> {
            String criteria = ((String)
searchSortCriteriaBox.getSelectedItem()).toLowerCase();
            int selectedAlgo = algoSelector.getSelectedIndex();

            if (selectedAlgo == 1 || criteria.equals("year")) {

SearchEngine.selectionSortByYear(manager.getItemsList());
                JOptionPane.showMessageDialog(this, "List
re-ordered using Selection Sort by Year.");
            } else {

```

```

        SearchEngine.mergeSort(manager.getItemsList(),
criteria);

        JOptionPane.showMessageDialog(this, "List
organized alphabetically using Merge Sort: " +
criteria.toUpperCase());
    }
    refreshTableCatalog();
});

addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        FileHandler.saveItems(manager.getItemsList());
    }
});

refreshTableCatalog();
}

private void refreshTableCatalog() {
    updateTableDataView(manager.getItemsList());
}

private void updateTableDataView(ArrayList<LibraryItem>
activeItems) {
    tableModel.setRowCount(0);
    for (LibraryItem item : activeItems) {
        tableModel.addRow(new Object[]{
            item.getId(),
            item.getItemType(),
            item.getTitle(),
            item.getAuthor(),
            item.getYear(),
            item.isBorrowed() ? "Checked Out By: " +
item.getCurrentBorrowerId() : "Available in Stack"
        });
    }
}
}
}

```

IDGenerator.java

package: **utils**

Java

```
public class IDGenerator {
    private static int itemCounter = 1000;

    public static synchronized String generateItemId(String type)
    {
        itemCounter++;
        return type.substring(0, 1).toUpperCase() + "-" +
itemCounter;
    }
}
```

FileHandler.java

package: **utils**

Java

```
import model.*;
import java.io.*;
import java.util.ArrayList;

public class FileHandler {
    private static final String FILE_PATH = "library_data.txt";

    public static void saveItems(ArrayList<LibraryItem> items) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(FILE_PATH))) {
            for (LibraryItem item : items) {
                String line =
String.format("%s,%s,%s,%s,%d,%b,%s",
                    item.getItemType(),
                    item.getId(),
                    item.getTitle(),
                    item.getAuthor(),
```



```

        item.getYear(),
        item.isBorrowed(),
        item.getCurrentBorrowerId().isEmpty() ?
"NONE" : item.getCurrentBorrowerId()
    );
    writer.write(line);
    writer.newLine();
}
} catch (IOException e) {
    System.err.println("Error saving data: " +
e.getMessage());
}
}

public static ArrayList<LibraryItem> loadItems() {
    ArrayList<LibraryItem> items = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new
FileReader(FILE_PATH))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] data = line.split(",");
            if (data.length < 7) continue;

            String type = data[0];
            String id = data[1];
            String title = data[2];
            String author = data[3];
            int year = Integer.parseInt(data[4]);
            boolean isBorrowed =
Boolean.parseBoolean(data[5]);
            String borrowerId = data[6].equals("NONE") ? "" :
data[6];

            LibraryItem item;
            switch (type) {
                case "Magazine": item = new Magazine(id,
title, author, year); break;
                case "Journal": item = new Journal(id, title,
author, year); break;
            }
        }
    }
}

```

```

        default: item = new Book(id, title, author,
year); break;
    }

    if (isBorrowed) {
        item.borrowItem(borrowerId);
    }
    items.add(item);
}
} catch (IOException e) {
    System.out.println("No existing registry file found.
Starting fresh.");
}
return items;
}
}

```

Main.java

package: **root**

```

Java
import gui.MainWindow;
import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MainWindow appWindow = new MainWindow();
            appWindow.setVisible(true);
        });
    }
}

```